OpenMP CSR SpMV 实验报告

1. 策略、环境与正确性

CSR 用 `row_ptr` 定位每行条目，计算独立的 `y[row]`。矩阵与 x 只读，每行由一个线程写回，累加变量私有，无需原子加法。

- **策略 1：逐行 guided，改善负载分配。**普通行与长行工作量差异大，固定均分行数容易让部分线程提前完成。外层使用 `schedule(guided)`，线程完成当前任务后继续领取连续行段；初期块较大，减少领取次数，后期逐步缩小，改善收尾阶段的负载分配。行内顺序累加，长行本身不拆分，因此不能保证工作量完全均衡。
- **策略 2：guided,16 + SIMD，结合线程并行与行内并行。**外层使用 `schedule(guided, 16)`，以最小 16 行的块粒度减少细粒度任务领取；尾块可少于 16 行。行内使用 `omp simd reduction(+:sum)`，允许编译器拆分累加并合并部分和，减少单一累加链的依赖。间接访问 x 和归约成本可能限制收益，故用固定 guided,16 的开关实验验证 SIMD 效果。
- **策略 3：64 行分块 guided，检验更粗的工作划分。**将连续 64 行作为一个外层迭代，guided 调度的是这些行块，初期一次可领取多个块；块内顺序计算，最后一块处理余数。设计意图是减少外层调度单元并保持连续遍历，但一个块中的长行不能再分给其他线程，可能加重收尾不均衡；是否获益由实测判断。

项目	配置
CPU 与系统	Apple M4，10 个物理核（4P + 6E），macOS 26.7.1
编译器与运行时	Homebrew Clang 23.1.2、LLVM libomp，OpenMP 202011
编译选项	`-O2 -std=c++17 -fopenmp -pthread`；采样版额外加 `-g`
默认矩阵	10000×100000，958976 个非零存储条目
行长与随机性	普通行各 32 个条目；32 个长行各 20000 个；种子 20260926
主基准配置	8 线程，`OMP_DYNAMIC=FALSE`，未显式绑定 P/E 核

同组使用相同输入、编译选项和计时范围。主基准独立运行 10 轮，轮换策略顺序；每个实现先预热 1 次并全量校验，再用 `steady_clock` 计时 5000 次，统计单次平均耗时的中位数与范围。生成、预热、校验和输出不计时。

保留串行参考，全量逐元素比较，绝对和相对误差容差均为 `1e-10`。SIMD 改变累加顺序，按相同容差检查；默认矩阵 1/2/4/8/10 线程及空行、单长行、分块尾部共 8 组均通过，截图见第 4 页。

2. 运行时间、加速比与并行效率

定义 T1 为本组串行中位耗时，加速比 S=T1/Tp，并行效率 E=S/p。下表为默认矩阵、8 线程、10 轮 ×5000 次的结果；其他配置各用自己的串行基线。

实现	中位耗时/ms	最小–最大/ms	加速比	效率
串行	0.748359	0.731852–0.757485	1.000×	—
std::thread	0.246725	0.239328–0.255293	3.033×	37.91%
逐行 guided	0.189346	0.170881–0.210680	3.952×	49.40%
guided,16 + SIMD	0.162950	0.146843–0.174546	4.593×	57.41%
64 行块 guided	0.192445	0.185624–0.207118	3.889×	48.61%

本次策略 2 最快，相对串行加速 **4.593 倍**。逐行 guided 比 std::thread 耗时低约 **23.3%**。64 行块 guided 未优于逐行 guided，增加块粒度未带来净收益。

3. std::thread 对比与 SIMD 消融

两者都适合按行独立计算。std::thread 基线按连续行分配，每次创建并 join 工作线程，短任务反复承担线程管理成本，且行数均分不保证工作量均衡。OpenMP 用指令控制并行循环、调度和线程数，运行时通常复用线程；std::thread 则便于自定义线程生命周期、任务队列与通信。本实验未测试线程池。

SIMD 消融固定 8 线程、**guided,16**、输入与编译选项，仅开关行内 SIMD。独立测量 10 轮 ×5000 次，轮换顺序，每次先预热 1 次并校验。

行内实现	中位耗时/ms	最小–最大/ms
顺序累加，关闭 SIMD	0.184491	0.171050–0.206685
SIMD 归约	0.161849	0.144538–0.169493

SIMD 使中位耗时降低 **12.27%**。该表单独测量；策略 1 与 2 还改变了 chunk，不能作为纯 SIMD 消融。

4. 参数实验与重复性

研究线程数、矩阵规模和 chunk 三类变量。前两类使用 guided 策略 1 的 10 轮 ×500 次记录，其生成器与内核已核对一致。每个配置先预热 1 次并校验。

线程数与矩阵规模

线程数实验固定默认矩阵；规模实验固定 8 线程、100000 列，行数与长行数同比变化，长行比例为 0.32%。

线程数	guided/ms	加速比	效率
1	0.732214	0.993×	99.29%
2	0.385775	1.896×	94.80%
4	0.207377	3.515×	87.87%
8	0.166548	4.417×	55.22%
10	0.166808	4.262×	42.62%

行数	非零条目数	guided/ms	加速比
2500	239744	0.067470	2.675×
10000	958976	0.166548	4.417×
40000	3835904	0.551362	5.318×

1→8 线程收益明显，8→10 基本持平。规模增大时加速比提高，与固定开销被摊薄相符。

guided 最小 chunk 与顺序复测

固定 4 线程、默认矩阵，10 轮 ×5000 次，每个配置预热 1 次并校验。每轮随机排列五个 chunk，最后测串行；两次使用相同排列种子 20261017。chunk 为最小块行数，尾块可更小。

chunk	随机第 1 次/ms	随机第 2 次/ms	第 2 次加速比
1	0.210475	0.217074	3.424×
16	0.213844	0.215183	3.454×
32	0.213050	0.219627	3.384×
64	0.216615	0.224274	3.314×
128	0.227619	0.227751	3.263×

两次串行基线为 0.740799、0.743180 ms。第一次 chunk=1 最快，第二次为 16；1/16/32 差距较小，128 两次均较慢。保留策略 1 的默认 chunk=1，不把单次排名视为普遍最优。

5. Instruments 性能分析

本机使用 Instruments 27.0 的 Time Profiler 和 CPU Counters，作为 macOS 下的替代分析；作业 3.5 指定的工具为 Linux perf。

串行、策略 1、std::thread 各录制一次 Time Profiler，8 线程、10000 次正式调用，先预热 1 次并校验。下表为整个进程（含准备阶段）的栈顶 Self 采样权重占比；耗时采用第 2 节未开采样器的结果。

实现	SpMV 核心 Self	观察到的运行时热点
串行	99.64%	热点主要在 SpMV 循环
guided	95.25%	任务分配函数合计约 1.01%，条件变量等待/通知约 2.66%
std::thread	90.62%	创建线程相关函数合计约 2.29%，另有等待及清理

std::thread 采样到 9315 个不同线程 ID，guided 为 8 个，支持线程反复创建回收的解释。该数量并非同时运行线程数；Self 占比不能视作墙钟开销。

CPU Counters 使用独立的 static、guided、dynamic,1 对照内核，4 线程、3 次 trace×10000 次调用，仅统计预热校验后的 MeasuredSpMV 区间。IPC 为采样值，L1D 指推测执行 Load Miss/Load µop；这些对照内核与提交策略分开统计。

对照调度	采样 IPC	L1D Load Miss/Load µop
static	1.989	32.14%
guided	1.986	31.51%
dynamic,1	2.153	29.20%

IPC 或 L1D 比值还受运行库指令与访存影响，不能单独判断快慢。未采集 L2 或实际 DRAM 流量，不据此认定缓存或带宽瓶颈。

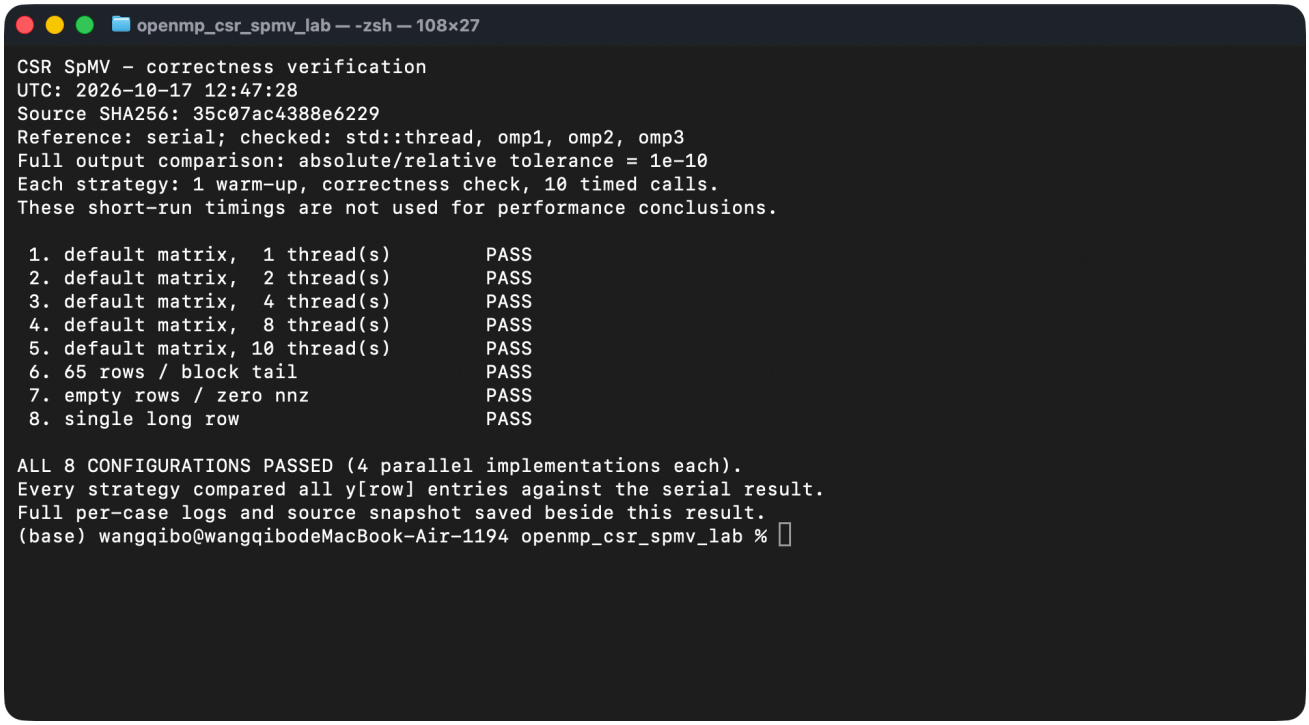
6. 结论、局限性与来源

本次优选 guided,16 + SIMD：相对串行加速 4.593 倍，固定调度参数的消融显示 SIMD 降低耗时 12.27%。64 行分块没有进一步提速。结论限于本机与该矩阵族；未固定 P/E 核、频率和温度，跨轮存在波动。扩展验证可增加偏斜分布、固定线程位置，或测试 std::thread 线程池。

AI 使用声明：Codex 辅助解释代码、编写测试和分析工具、整理报告；数据来自实际运行与 trace，需本人核实。个人过程记录由本人撰写，Codex 辅助调整语序与格式。

附录 A：正确性结果真实截图

以下截图来自实际运行验证工具的 macOS Terminal 窗口，非重绘日志。8 个配置均对完整输出逐元素校验；每个配置包含 std::thread 和三个 OpenMP 实现，使用同一串行参考。



真实终端正确性检查：8 个配置全部通过

对应源码 SHA256：`35c07ac4388e62290b87f4e032dca66c653a27678e28d8d7688f7cc0b5333440`。

完整日志位于截图目录的 `logs/`，含 `results.json` 和 `case_1.txt` 至 `case_8.txt`。这些短运行仅用于正确性，不用于性能结论。

附带数据：主基准 `benchmark_three/`、消融 `simd_ablation/`、Time Profiler `instruments/`；线程数与规模见 `output/parameter_results/`，chunk 两次随机复测见对应目录。历史完整探索报告已移至 `output/final_report/report_before_completion.md`，不作为当前版本结论的依据。